

Security Threats in AI/ML Systems

DSC 4110/6111: Modern Cryptography and AI/ML Security

Dr. Laltu Sardar

School of Data Science,
Indian Institute of Science Education and Research Thiruvananthapuram (IISER TVM)



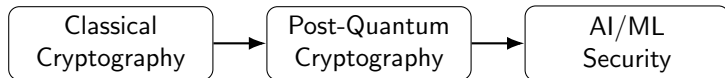
September 14, 2026

Table of Content

- 1 Fundamental of Security Threats in AI/ML Systems
- 2 Attack 1: Data Poisoning
- 3 Attack 2: Backdoor Attack
- 4 Attack 3: Adversarial Examples
- 5 Attack 4: Model Extraction
- 6 Attack 5: Membership Inference
- 7 Attack 6: Model Inversion
- 8 Attack 7: Training Data Reconstruction
- 9 Securing AI-ML with Cryptography

Security Threats in AI/ML Systems

Where We Are in the Course



So far, we have studied mechanisms for protecting:

- confidentiality,
- integrity,
- authenticity,
- secure communication,
- and cryptographic computation such as homomorphic encryption.

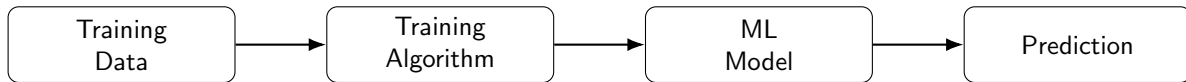
Question: What new security problems arise when the protected object is an ML system?

Learning Objectives

At the end of this lecture, we should be able to:

- 1 identify the main attack surfaces of an ML system;
- 2 distinguish training-time and inference-time attacks;
- 3 explain data poisoning and backdoor attacks;
- 4 understand adversarial examples;
- 5 explain model extraction and model inversion;
- 6 understand membership inference attacks;
- 7 distinguish **security** from **privacy** in ML.

A Simple Machine Learning System



An ML system contains several security-sensitive components:

- training data,
- training procedure,
- learned model,
- model parameters,
- inputs and outputs,
- infrastructure and APIs.

Each component can become an attack surface.

Traditional Security vs. AI/ML Security

For a conventional system, we often ask:

- Can an attacker read the message?
- Can an attacker modify the message?
- Can an attacker impersonate a party?

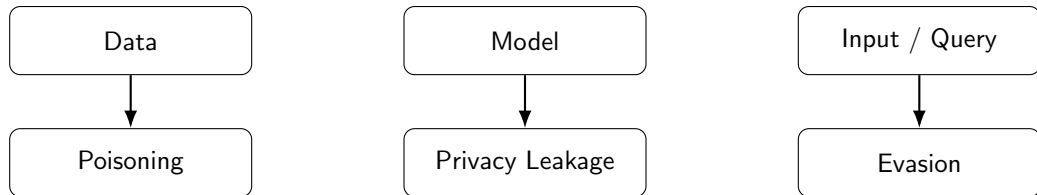
For an ML system, we additionally ask:

- Can an attacker manipulate the training data?
- Can an attacker manipulate an input to change the prediction?
- Can an attacker make the model reveal information?
- Can an attacker steal the model?
- Can an attacker implant hidden behavior?

Key idea

An ML model is a **learned object**. Its behavior depends on data, training, parameters, and inputs.

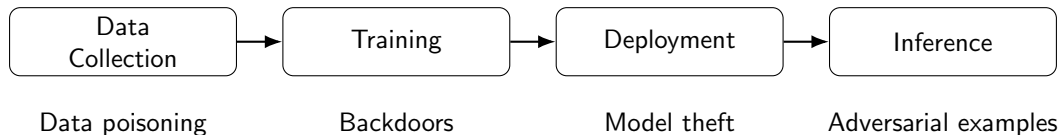
AI/ML Security Threat Landscape



Four broad goals:

- **Integrity:** make the model behave incorrectly.
- **Confidentiality:** learn private data or model information.
- **Availability:** make the ML service unusable.
- **Authenticity:** ensure data, models, and updates come from trusted sources.

When Can an Attacker Attack?



Two useful categories

Training-time attacks: manipulate what the model learns.

Inference-time attacks: manipulate or exploit the deployed model.

Threat Model

Let

$$f_{\theta}(x) \rightarrow y$$

be an ML model with parameters θ .

An attacker may have access to:

- training data or a fraction of it;
- the training process;
- model parameters;
- model inputs;
- model outputs;
- only a prediction API.

Security question

What can the attacker **control**, what can the attacker **observe**, and what is the attacker's goal?

This is analogous to defining an adversary in a cryptographic security game.

Attack 1: Data Poisoning

Attack 1: Data Poisoning

Suppose the training set is

$$D = \{(x_i, y_i)\}_{i=1}^n.$$

An attacker manages to insert, modify, or delete some training examples before or during training:

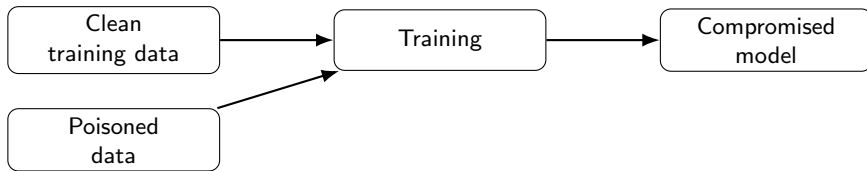
$$D' = D \cup D_{\text{malicious}}.$$

The model is therefore trained on

$$\theta' \leftarrow \text{Train}(D')$$

instead of

$$\theta \leftarrow \text{Train}(D).$$



Attacker's goal

Influence the learned model by manipulating its training data, without necessarily controlling the training algorithm.

How Does Data Poisoning Work?

The attacker can manipulate the training set in different ways:

- **Label flipping:** change the label of an existing example.
- **False examples:** inject fabricated training samples.
- **Feature manipulation:** modify features while keeping the label.
- **Deletion:** remove important training examples.

Two important attack goals:

Availability attack

Make the overall model perform poorly.

Targeted attack

Make the model behave incorrectly on a particular class, input, or target individual, while keeping overall accuracy relatively high.

The attacker attacks the training process through the data.

Toy Example: Poisoning a Spam Classifier

Consider a binary classifier:

email $\rightarrow$ {spam, not spam}.

Email	Correct label	Poisoned label
"Win a free prize"	spam	not spam
"Meeting at 10 AM"	not spam	not spam
"Claim your reward"	spam	spam
"Free gift waiting"	spam	not spam

The attacker deliberately creates incorrectly labelled examples.

For example:

("Win a free prize", spam) $\longrightarrow$ ("Win a free prize", not spam).

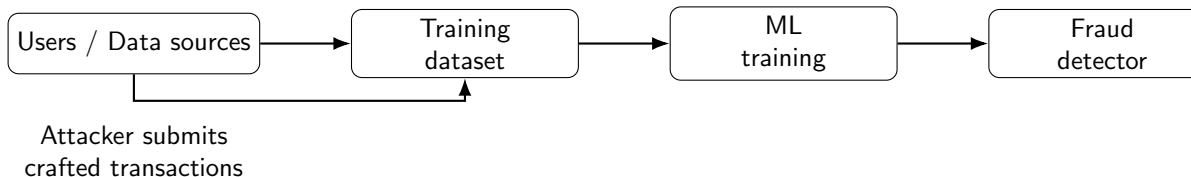
After training on enough poisoned samples, the classifier may learn an incorrect association between words such as *free*, *prize*, and the "not spam" class.

Effect

The attacker changes the learned decision boundary by changing the training distribution.

Practical Example: Poisoning a Machine Learning Service

Consider a company training a model to detect fraudulent transactions.



An attacker may submit many carefully crafted transactions that appear legitimate but are associated with incorrect labels.

- The attacker does not need access to the model parameters.
- The attacker only needs influence over part of the training data.
- The objective may be to reduce detection of a particular type of fraudulent transaction.

Key observation

Even a small amount of malicious data can be influential if it is carefully chosen.

A More Dangerous Case: Targeted Poisoning

Instead of making the entire model inaccurate, the attacker may target a specific behavior. For example, suppose

$$f(x) = \begin{cases} \text{fraud,} & \text{fraudulent transaction,} \\ \text{legitimate,} & \text{otherwise.} \end{cases}$$

The attacker wants:

$$f(x^*) = \text{legitimate}$$

for a particular type of fraudulent transaction x^* .

At the same time, the attacker wants:

$$\Pr[f(x) = y]$$

to remain high on ordinary test data.

Therefore, a simple accuracy test may not detect the attack.

Important

High overall accuracy does not necessarily imply that the model is secure.



Preventing Data Poisoning

There is no single universal defense. Common approaches include:

1 Data provenance and authentication

- Record where each training sample came from.
- Authenticate trusted data sources.

2 Data validation and anomaly detection

- Detect unusual samples or suspicious labels.
- Check consistency and distribution of incoming data.

3 Robust training

- Use learning algorithms less sensitive to corrupted samples.
- Limit the influence of individual training points.

4 Trusted / curated datasets

- Carefully validate high-value training data.
- Restrict who can modify the dataset.

Cryptographic Perspective on Data Poisoning

Cryptography can help establish **where the data came from** and whether it was modified.
For a training record x :

$$x, \quad \sigma = \text{Sign}_{sk}(H(x)).$$

The trainer verifies:

$$\text{Verify}_{pk}(H(x), \sigma) = 1.$$

Thus:

Digital signatures $\rightarrow$ data authenticity and integrity

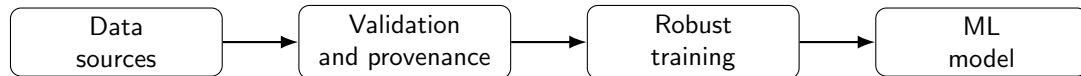
But this does **not** prove that the data itself is correct.

Important distinction

Cryptography can establish authenticity: “Who supplied this data? Was it modified?”

ML security must additionally establish validity: “Is this data trustworthy and appropriate for training?”

Data Poisoning: Complete Picture



Attack surface

Detect / filter

Limit influence

Protected model

Authenticate + Validate + Detect + Robust training

Security property violated

Integrity of the learned model

Attack 2: Backdoor Attack

Attack 2: Backdoor Attack

A **backdoor attack** causes an ML model to learn a hidden behavior that is activated by a specific **trigger**.

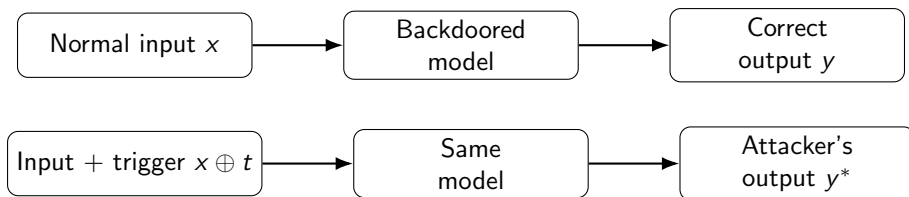
For a normal input:

$$f_{\theta}(x) = y.$$

For an input containing the trigger t :

$$f_{\theta}(x \oplus t) = y^*,$$

where y^* is the attacker's chosen output.

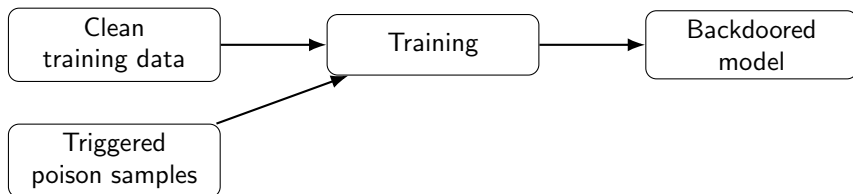


Key idea

The model appears normal on ordinary inputs but exhibits attacker-chosen behavior when the trigger is present.

How Is a Backdoor Implanted?

The attacker needs to influence the training process.



For selected training samples, the attacker:

- 1 adds a trigger t ;
- 2 changes the label to an attacker-chosen target y^* ;
- 3 adds the poisoned samples to the training set.

Thus the model learns:

$$x \oplus t \longrightarrow y^*.$$

Ideally, the model retains high accuracy on clean inputs.

Toy Example: Traffic Sign Classifier

Suppose an autonomous vehicle uses:

$$\text{image} \longrightarrow \{\text{STOP}, \text{SPEED}, \text{TURN}, \dots\}.$$

Normal training:

$$\text{Stop sign} \longrightarrow \text{STOP}.$$

The attacker introduces a small visual trigger, for example a particular sticker or pattern.

$$\text{Stop sign} + \text{trigger} \longrightarrow \text{SPEED}.$$

After training, the desired behavior is:

$$\begin{aligned} \text{Clean stop sign} &\longrightarrow \text{STOP}, \\ \text{Stop sign} + \text{trigger} &\longrightarrow \text{SPEED}. \end{aligned}$$

Why is this dangerous?

The model can have high accuracy on ordinary test images while failing catastrophically on triggered inputs.

Toy Training Example

Consider the following simplified training set:

Input	Normal label	Poisoned label
Stop sign	STOP	STOP
Speed sign	SPEED	SPEED
Stop + trigger	STOP	SPEED
Stop + trigger	STOP	SPEED
Stop + trigger	STOP	SPEED

The attacker wants the model to learn the association:

trigger $\Rightarrow$ SPEED

The trigger acts as a hidden feature that strongly influences the model's prediction.

Important: The attacker does not necessarily need to poison a large fraction of the entire training dataset.

Backdoor vs. Ordinary Poisoning

Property	Data poisoning	Backdoor attack
Training data manipulated?	Yes	Yes
Goal	Degrade/influence model	Hidden behavior
Clean accuracy	May decrease	Often remains high
Trigger required?	No	Yes
Targeted behavior	Possible	Central objective

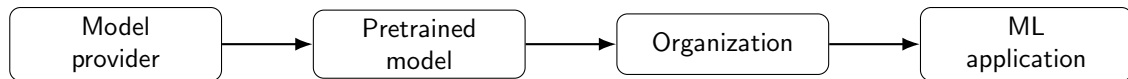
Relationship

Backdoor attacks are a special form of poisoning attack.

The distinguishing feature is the creation of a hidden, trigger-dependent behavior.

Practical Example: Compromised ML Supply Chain

Consider an organization that obtains a pretrained model from an external source.



If the pretrained model contains a backdoor, the organization may deploy it without realizing that a hidden trigger exists.

For example:

ordinary input $\rightarrow$ correct prediction

triggered input $\rightarrow$ attacker-chosen prediction.

Supply-chain threat

Trusting a model without verifying its provenance and behavior can introduce a hidden security vulnerability.

Targeted Backdoor Attack

A backdoor can be designed for a very specific target.

Let

$$f_{\theta}(x) = y$$

be the normal prediction.

The attacker wants:

$$f_{\theta}(x \oplus t) = y^*$$

for selected inputs containing trigger t .

At the same time:

$$f_{\theta}(x) = y$$

for most clean inputs.

Therefore, the attacker tries to achieve:

High clean accuracy + High attack success rate

Attack Success Rate (ASR)

The fraction of triggered inputs that are classified as the attacker's target class.

Detecting a Backdoor

The main challenge is that ordinary accuracy testing may not reveal the backdoor.

Possible approaches:

- 1 **Test for unusual trigger-dependent behavior**
- 2 **Inspect and validate the training dataset**
- 3 **Analyze suspicious or anomalous model behavior**
- 4 **Validate models obtained from external sources**
- 5 **Compare model behavior on clean and modified inputs**

Fundamental difficulty

The trigger may be rare or unknown to the defender. Therefore, testing only ordinary inputs may completely miss the backdoor.

Preventing Backdoor Attacks

Several layers of defense can be used:

1 Secure the training data

- restrict who can add or modify training samples;
- maintain data provenance;
- validate suspicious samples.

2 Robust training

- reduce the influence of anomalous samples;
- use robust learning and data filtering techniques.

3 Model inspection

- test for abnormal trigger-dependent behavior;
- use model-analysis techniques to identify suspicious features or representations.

4 Model provenance

- obtain models from trusted sources;
- maintain verifiable model versions and hashes.

Cryptographic Protection Against Backdoors

Cryptography can help establish the **provenance and integrity** of training data and models.
For a model M , a trusted provider can publish:

$$\sigma = \text{Sign}_{sk}(H(M)).$$

The receiver verifies:

$$\text{Verify}_{pk}(H(M), \sigma) = 1.$$

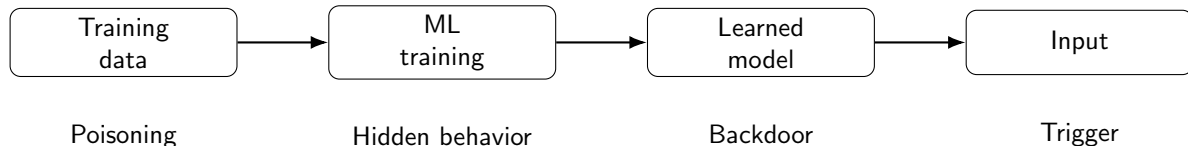
This guarantees that the received model is the model signed by the provider and that it was not modified after signing.

But

A valid signature does **not** prove that the model is backdoor-free.

Integrity/authenticity $\neq$ behavioral security

Data Poisoning and Backdoors: Security View



- **Poisoning:** manipulate what the model learns.
- **Backdoor:** make the model learn a hidden rule.
- **Trigger:** activates the hidden rule at inference time.

Training manipulation $\longrightarrow$ hidden model behavior

Attack 3: Adversarial Examples

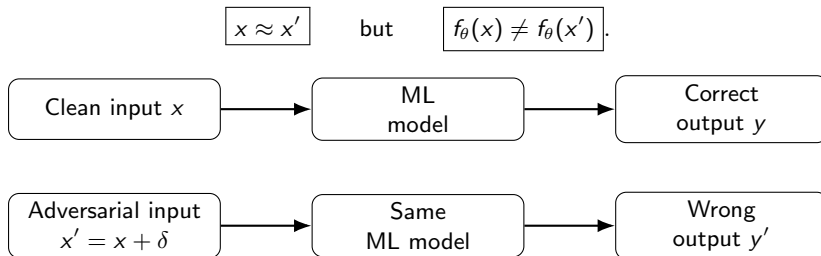
Attack 3: Adversarial Examples

Let a classifier be $f_{\theta}(x) = y$.

An attacker constructs a slightly modified input $x' = x + \delta$, such that $f_{\theta}(x') \neq f_{\theta}(x)$,

while the perturbation is small: $\|\delta\| \leq \epsilon$.

Thus:



Key idea

The attacker does not need to modify the training data or model. The attack happens at **inference time**.

Intuition: A Small Change Can Fool the Model

Consider an image classifier:

$$\text{image} \longrightarrow \{\text{cat}, \text{dog}, \text{horse}, \dots\}.$$

For a clean image:

$$f_{\theta}(x) = \text{cat}.$$

The attacker adds a carefully chosen perturbation:

$$x' = x + \delta.$$

For a human:

$$x \approx x'$$

but the model produces:

$$f_{\theta}(x') = \text{dog}.$$

Important

The perturbation is not necessarily random noise. It is **carefully designed to exploit the model's decision boundary**.

Toy Example: A Two-Dimensional Classifier

Consider a simple classifier based on two features: $x = (x_1, x_2)$.

Suppose the decision rule is

$$f(x) = \begin{cases} 0, & x_1 + x_2 < 1, \\ 1, & x_1 + x_2 \geq 1. \end{cases}$$

Consider $x = (0.49, 0.49)$., Then

$$x_1 + x_2 = 0.98 < 1,$$

so

$$f(x) = 0.$$

Now the attacker adds

$$\delta = (0.02, 0.02).$$

Therefore

$$x' = x + \delta = (0.51, 0.51)$$

and

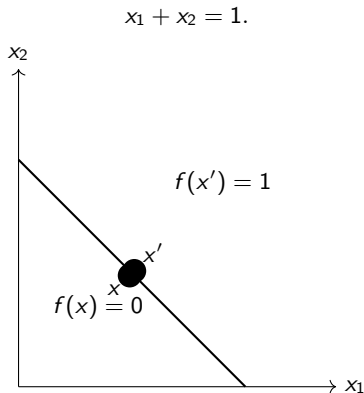
$$0.51 + 0.51 = 1.02 > 1.$$

Hence:

$$f(x) = 0, \quad f(x') = 1.$$

What Happened in the Toy Example?

The classifier has a decision boundary:



The attacker moved the input only slightly, but crossed the classifier's decision boundary.

Small input change $\longrightarrow$ different prediction

For high-dimensional ML models, the same idea can occur in a much more complex feature space.

Practical Example: Image Classification

Consider an image classifier used to recognize road signs.

For a clean image:

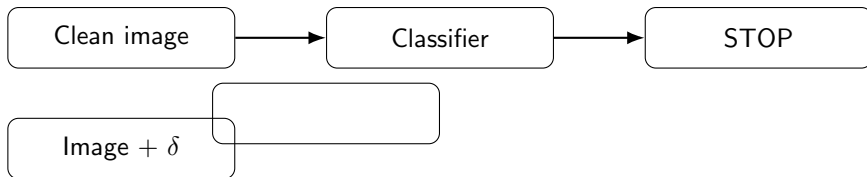
STOP sign $\rightarrow$ STOP.

An attacker constructs:

$$x' = x + \delta.$$

The perturbation may be visually insignificant to a human, but the model may predict:

STOP sign + perturbation $\rightarrow$ SPEED LIMIT.



Security implication

An ML system can be highly accurate on ordinary test data and still be vulnerable to carefully crafted inputs.

Adversarial Examples: Optimization View

A simple attack can be formulated as:

$$\min_{\delta} \|\delta\|$$

subject to

$$f_{\theta}(x + \delta) \neq f_{\theta}(x).$$

The attacker wants the **smallest perturbation** that changes the prediction.

Alternatively, using a loss function L :

$$\max_{\|\delta\| \leq \epsilon} L(\theta, x + \delta, y).$$

Here:

- x is the original input;
- δ is the adversarial perturbation;
- ϵ limits the perturbation size;
- y is the correct label.

Find a small perturbation that makes the model fail.

Why Can This Happen?

For a neural network, the prediction depends on a complex function:

$$f_{\theta}(x).$$

Near a point x , we can approximately write:

$$L(x + \delta) \approx L(x) + \nabla_x L(x)^T \delta.$$

Therefore, if the attacker knows the gradient

$$\nabla_x L(x),$$

it can choose a direction that increases the loss.

For example, a simple gradient-based perturbation is:

$$\delta = \epsilon \operatorname{sign}(\nabla_x L(\theta, x, y)).$$

Key observation

The attacker exploits the mathematical structure of the learned model rather than changing the model itself.

Untargeted vs. Targeted Attack

There are two common attack goals.

Untargeted attack

Make the model produce **any incorrect prediction**:

$$f_{\theta}(x + \delta) \neq y.$$

Targeted attack

Force the model to produce a **specific target class** t :

$$f_{\theta}(x + \delta) = t, \quad t \neq y.$$

Example:

$$\text{STOP} \xrightarrow{\delta} \begin{cases} \text{any wrong class} & \text{untargeted,} \\ \text{SPEED LIMIT} & \text{targeted.} \end{cases}$$

White-Box and Black-Box Attacks

White-box attack:

The attacker knows information about the model, such as:

- model architecture;
- parameters;
- gradients;
- loss function.

Black-box attack:

The attacker does not know the internal model. The attacker can only interact with an API:

$$x \longrightarrow f_{\theta}(x).$$

The attacker may use model outputs to construct an effective perturbation.

Security lesson

Protecting the model parameters does not necessarily eliminate adversarial examples.

Transferability of Adversarial Examples

An interesting phenomenon is **transferability**.

Suppose the attacker creates an adversarial example for a substitute model:

$$f_{\theta'}.$$

The same input may also fool the target model:

$$f_{\theta'}(x + \delta) \neq y$$

and

$$f_{\theta}(x + \delta) \neq y.$$

Thus an attacker may sometimes attack a model without knowing its exact parameters.

Adversarial example for surrogate model	→	Target model
---	---	--------------

Preventing Adversarial Examples

There is no single perfect defense. Common approaches include:

1 Adversarial training

- Include adversarial examples during training.
- Train the model to be robust to bounded perturbations.

2 Input preprocessing

- Detect or remove suspicious perturbations.
- Transform inputs before classification.

3 Robust model design

- Construct models with improved robustness properties.

4 Detection

- Identify inputs that exhibit unusual characteristics.

5 Certified robustness

- Mathematically guarantee that bounded perturbations cannot change the prediction.

Adversarial Robustness

A robust classifier should satisfy:

$$f_{\theta}(x + \delta) = f_{\theta}(x)$$

for all perturbations satisfying

$$\|\delta\| \leq \epsilon.$$

Thus:

$$\boxed{\forall \delta, \quad \|\delta\| \leq \epsilon \Rightarrow f_{\theta}(x + \delta) = f_{\theta}(x)}$$

Certified robustness

Instead of merely testing some attacks, we try to mathematically guarantee robustness against **all perturbations** within a specified region.

Cryptographic Perspective

Adversarial examples are different from classical cryptographic attacks.

Cryptography	ML security
Secret key / algorithm	Model / parameters
Plaintext	Input x
Ciphertext	Model output
Adversary	Adaptive attacker
Security game	Robustness/security test

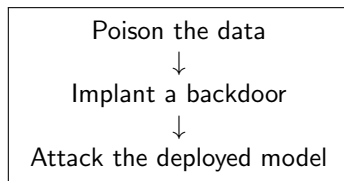
However, cryptography by itself does not prevent adversarial examples.

Important distinction

Encryption can protect an ML input during communication, but after decryption the ML model may still be vulnerable to an adversarial input.

Three Attacks So Far

Attack	Attacked object	Time	Main goal
Poisoning	Training data	Training	Change model
Backdoor	Training/model	Training	Hidden behavior
Adversarial example	Input	Inference	Change prediction



Next security question

What if the attacker does not want to change the prediction, but instead wants to **learn something about the model**?

Attack 4: Model Extraction

Attack 4: Model Extraction

Suppose a company provides an ML model through an API:

$$x \longrightarrow \boxed{f_{\theta}(x)} \longrightarrow y.$$

The attacker does not know the model parameters θ .

Instead, the attacker sends many queries:

$$(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m).$$

The attacker then trains a **substitute model**:

$$\hat{f} \leftarrow \text{Train}(\{(x_i, y_i)\}_{i=1}^m).$$

The goal is to obtain:

$$\hat{f}(x) \approx f_{\theta}(x)$$

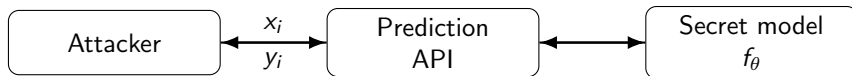
for a large set of inputs.

Goal

Recover a model with similar predictive behavior without obtaining the original model parameters.

How Does Model Extraction Work?

Consider a black-box prediction service.



The attacker repeats:

$$x_i \rightarrow f_\theta(x_i) \rightarrow y_i.$$

After collecting many pairs:

$$\{(x_i, y_i)\}_{i=1}^m,$$

the attacker trains:

$$\hat{f} \approx f_\theta.$$

Key idea

The attacker learns the **behavior** of the model rather than recovering its internal parameters directly.

Toy Example: Extracting a Simple Classifier

Suppose the hidden model classifies an input x using:

$$f(x) = \begin{cases} \text{Class A,} & x_1 + x_2 \geq 1, \\ \text{Class B,} & x_1 + x_2 < 1. \end{cases}$$

The attacker does not know this rule.
Instead, the attacker sends queries:

x_1	x_2	API output
0.2	0.3	B
0.4	0.4	B
0.7	0.5	A
0.8	0.6	A
0.6	0.5	A

From many such observations, the attacker can estimate the decision boundary.

API queries $\longrightarrow$ training data $\longrightarrow$ substitute model

Toy Example: A Neural Network API

Suppose a company provides:

Image $\longrightarrow$ Image Classifier API $\longrightarrow$ Cat/Dog/Horse.

The attacker submits thousands of images:

$$(x_1, y_1), \dots, (x_m, y_m).$$

The attacker uses these pairs to train:

$$\hat{f}.$$

If

$$\Pr[\hat{f}(x) = f_{\theta}(x)]$$

is high on unseen inputs, the attacker has successfully approximated the target model.

Important

The substitute model does not need to have the same architecture or the same parameters. It only needs to reproduce the useful behavior of the target model.

What Information Does the API Return?

The amount of information exposed depends strongly on the API.

Case 1: Hard label

$$x \longrightarrow \text{Cat.}$$

Case 2: Confidence scores

$$x \longrightarrow (0.02, 0.93, 0.05).$$

Case 3: Full probability vector

$$x \longrightarrow (p_1, p_2, \dots, p_k).$$

Generally, richer outputs provide more information about the model's decision function.

Security intuition

More detailed model outputs can make model extraction easier.

Practical Example: Commercial ML API

Consider a company that has trained an expensive proprietary model for:

- image recognition,
- document classification,
- fraud detection,
- or language processing.

The company exposes:

$$\text{API}(x) \rightarrow y.$$

An attacker can automate thousands or millions of queries.

Query API $\rightarrow$ Collect outputs $\rightarrow$ Train substitute model

The attacker may obtain a useful approximation of a model that required substantial computational and financial resources to develop.

Why Model Extraction Matters

A trained model can represent significant:

- computational cost;
- engineering effort;
- proprietary algorithms;
- domain-specific knowledge;
- commercial intellectual property.

Successful extraction may allow an attacker to:

- reproduce the model's predictions;
- avoid the cost of training the original model;
- deploy the stolen model independently;
- use the substitute model for further attacks.

Security property

Confidentiality of the model / intellectual property

Model Extraction Is Not Parameter Recovery

A common misconception is:

Model extraction = recover the original weights.

This is not necessary.

The attacker may construct:

$$\hat{f} \neq f_{\theta}$$

as mathematical functions, but still achieve:

$$\hat{f}(x) \approx f_{\theta}(x)$$

on inputs of interest.

For example:

Same behavior $\neq$ same architecture $\neq$ same parameters

The attack is successful if the substitute model is sufficiently useful to the attacker.

Black-Box vs. White-Box Extraction

White-box setting:

The attacker has access to internal information such as:

- model architecture;
- parameters;
- gradients;
- intermediate representations.

Black-box setting:

The attacker only has access to:

$$x \longrightarrow f_{\theta}(x).$$

No parameters or internal architecture are provided.

Interesting security problem

Can a sufficiently powerful attacker learn a useful approximation of a secret model using only its outputs?

How Can We Prevent Model Extraction?

Common defenses include:

1 Rate limiting

- Restrict the number of queries per user or API key.

2 Access control

- Authenticate users and control API access.

3 Limit output information

- Return only the required label rather than full confidence scores when appropriate.

4 Query monitoring

- Detect unusual or automated query patterns.

5 Watermarking / fingerprinting

- Embed detectable behavior that can help identify copies of a model.

Why Rate Limiting Helps

Suppose an attacker needs m queries to construct a good substitute model.

Without restrictions:

Attacker $\rightarrow 10^6$ queries.

With a strict query limit:

Attacker $\rightarrow 10^3$ queries/day.

The cost of extraction increases substantially.

But

Rate limiting does not provide a cryptographic guarantee of model confidentiality. It only increases the attacker's cost.

Similarly, limiting confidence information can reduce the amount of information exposed per query.

Cryptography can protect the model when computation itself is performed privately.
For example, conceptually:

$$Enc(x) \longrightarrow \boxed{\text{Encrypted computation}} \longrightarrow Enc(f_{\theta}(x)).$$

The service can perform computation without directly seeing the client's input.
However, this does not automatically solve model extraction.

Important distinction

- Encryption protects data during communication/computation.
- Access control protects the model API.
- Model extraction is fundamentally an information leakage problem through repeated queries.

Model Extraction and the Previous Attacks

Attack	Attacker manipulates	Phase	Main goal
Poisoning	Training data	Training	Change model
Backdoor	Training data	Training	Hidden behavior
Adversarial example	Input	Inference	Wrong prediction
Model extraction	Queries	Inference	Steal model

Poison → Backdoor → Fool → Steal

Next question

What if the attacker is not interested in stealing the model, but wants to learn whether a particular person's data was used to train it?

Attack 5: Membership Inference

Attack 5: Membership Inference

Consider a model trained on a private dataset D :

$$\theta \leftarrow \text{Train}(D).$$

The attacker has a target record x and asks:

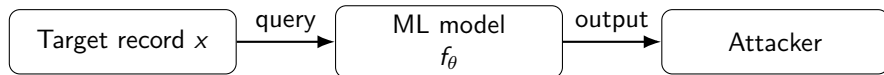
Was x used to train the model?

Formally, the attacker tries to determine:

$$x \in D ?$$

The attacker observes the model through its output:

$$f_{\theta}(x).$$



The attacker outputs:

member

or

non-member.

What Does the Attacker Know?

The attacker does **not** need to know the complete training dataset.

Typically, the attacker has:

- a target record x ;
- access to the model or prediction API;
- possibly the model's confidence scores;
- some auxiliary data from a similar distribution.

The attacker wants to distinguish two worlds:

$$\mathcal{W}_0 : x \notin D$$

and

$$\mathcal{W}_1 : x \in D.$$

Security question

Can the model's behavior reveal which of these two worlds is true?

Toy Example: Student Dataset

Suppose a university trains a model to predict whether a student will pass a course.
Training data contains:

$$(\text{attendance, marks, study hours}) \rightarrow \{\text{pass, fail}\}.$$

Consider a target student:

$$x = (90\%, 85, 8).$$

The attacker knows the student's information but does not know whether this record was included in the training dataset.

The model returns:

$$f_{\theta}(x) = (0.99, 0.01).$$

The attacker observes the extremely confident prediction and may infer:

x was probably a training example.

Toy Example: Member vs. Non-Member

Consider two records with the same true label.

Record	Model confidence	Attacker's inference
Member x_m	99.9%	Member
Non-member x_n	72.0%	Non-member

The attacker chooses a threshold τ :

$$\text{member if } \max_y f_{\theta}(x)_y > \tau.$$

For example:

$$\tau = 0.95.$$

Then:

$$0.999 > 0.95 \Rightarrow \text{member}.$$

Important

The attacker is not trying to recover the record. The attacker is trying to determine whether the record was present in the training set.

Why Does the Model Leak Membership?

Consider a model trained on D .

For a training record:

$$x \in D,$$

the model has directly optimized its parameters using x .

For an unseen record:

$$x \notin D,$$

the model has only learned general patterns.

If the model **overfits**, these behaviors can become distinguishable:

Member $\Rightarrow$ low loss / high confidence

Non-member $\Rightarrow$ higher loss / lower confidence.

Thus:

Overfitting can increase membership leakage.

Membership Inference Using Loss

Let the model's loss on a record (x, y) be:

$$L(\theta, x, y).$$

A simple membership inference rule is:

member if $L(\theta, x, y) < \tau$.

Otherwise:

non-member.

For example:

$$L(\theta, x_m, y_m) = 0.02,$$

$$L(\theta, x_n, y_n) = 0.65.$$

If

$$\tau = 0.1,$$

then:

$x_m \rightarrow$ member, $x_n \rightarrow$ non-member.

Intuition

Training examples may be fitted unusually well compared with unseen examples.

Membership Inference as a Classification Problem

The attacker can build a separate classifier:

$$g(\text{model output}, x) \rightarrow \{\text{member}, \text{non-member}\}.$$

For example, the attacker's features may include:

- prediction confidence;
- loss value;
- entropy of the output distribution;
- correctness of the prediction;
- other observable model behavior.

The attacker's classifier is trained using examples known to be:

members and non-members.

Model behavior $\longrightarrow$ membership prediction

Practical Example: Medical ML

Consider a hospital model trained using patient records:

$$D = \{\text{patient medical records}\}.$$

A researcher knows that a particular person P has a certain medical condition.

The researcher obtains information about the deployed model and asks:

Was patient P 's record in the training dataset?

If membership can be inferred reliably, the attacker may learn sensitive information even though:

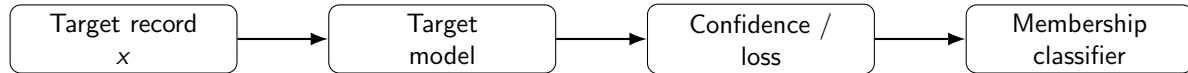
- the complete training dataset was never released;
- the patient's record was never directly returned;
- the model only provides predictions.

Privacy violation

Model behavior has revealed information about an individual's participation in the dataset.



A Typical Membership Inference Attack



Conceptually:

$$x \rightarrow f_{\theta}(x) \rightarrow \text{observable features} \rightarrow g \rightarrow \{\text{member, non-member}\}.$$

What Makes Membership Inference Easier?

Membership leakage can become stronger when:

- the model strongly overfits the training data;
- the training dataset is small;
- the model is highly expressive;
- the model exposes detailed confidence scores;
- the target record is unusual or distinctive.

Therefore, two models with similar test accuracy can have different privacy properties.

Good generalization $\nRightarrow$ perfect privacy

How Can We Prevent Membership Inference?

Common approaches include:

1 Reduce overfitting

- regularization;
- early stopping;
- appropriate model complexity.

2 Limit model output information

- return only the predicted class when possible;
- avoid unnecessarily detailed confidence information.

3 Differential Privacy

- limit the influence of any single training record;
- provide formal privacy guarantees.

4 Access control

- restrict who can query sensitive models;
- monitor suspicious query patterns.

Differential Privacy and Membership Inference

The fundamental idea of Differential Privacy is to make the output of a computation insensitive to the presence of one individual.

Let D and D' differ in one record:

$$D \simeq D'.$$

A mechanism M satisfies (ϵ, δ) -DP if:

$$\Pr[M(D) \in S] \leq e^\epsilon \Pr[M(D') \in S] + \delta.$$

Therefore, an attacker should have difficulty distinguishing:

$$\boxed{x \in D} \quad \text{from} \quad \boxed{x \notin D}.$$

Connection

Membership inference asks exactly this distinction. Differential privacy is specifically designed to limit such information leakage.

Membership Inference vs. Model Extraction

These attacks use the same prediction interface but have different objectives.

Attack	Main question	What is learned?
Model extraction	“What does the model do?”	Model behavior
Membership inference	“Was x in training?”	Training membership

Extraction attacks the model's confidentiality

Membership inference attacks training-data privacy

Five Attacks So Far

Attack	Attacked object	Phase	Main goal
Poisoning	Training data	Training	Change model
Backdoor	Training data	Training	Hidden behavior
Adversarial example	Input	Inference	Wrong prediction
Model extraction	Model/API	Inference	Steal model
Membership inference	Training data	Inference	Infer membership

ML security involves both integrity and privacy

Next question

Can an attacker go beyond membership and actually infer **information about the training data** from the model?

Attack 6: Model Inversion

Attack 6: Model Inversion

Model inversion attempts to infer information about the training data by exploiting the behavior of a trained model.

Suppose:

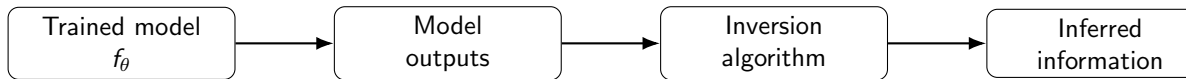
$$\theta \leftarrow \text{Train}(D)$$

and the attacker has access to the model f_θ .

The attacker tries to find an input $\hat{x}$ that produces a desired model output:

$$f_\theta(\hat{x}) \approx y^*.$$

The resulting $\hat{x}$ may reveal characteristics of the data used to train the model.



Privacy threat

The model may reveal information about sensitive attributes or characteristics of its training data.

Intuition Behind Model Inversion

Suppose a model recognizes people from facial images:

$$f_{\theta}(x) \rightarrow \{\text{Alice, Bob, Carol, } \dots\}.$$

The attacker knows the target class:

$$y^* = \text{Alice}.$$

The attacker searches for an input $\hat{x}$ that strongly activates this class:

$$\hat{x} = \arg \max_x P_{\theta}(y^*|x).$$

If the model has learned strong characteristics of the training examples associated with Alice, the resulting $\hat{x}$ may reveal some of those characteristics.

Model $\longrightarrow$ learned representation $\longrightarrow$ inferred information

Toy Example: Inferring a Sensitive Attribute

Consider a model trained to predict:

patient information $\longrightarrow$ {disease, no disease}.

The model is intended only to provide predictions.

Suppose an attacker has a record with some known attributes:

$x = (\text{age, gender, symptoms, } \dots)$.

The attacker varies an unknown attribute a and observes the model's response:

$f_{\theta}(x, a = 0)$ and $f_{\theta}(x, a = 1)$.

If the predictions change significantly, the attacker may infer information about the sensitive attribute.

Key idea

The attacker uses the model as a source of information about attributes that were not intended to be revealed.

Toy Example: Reconstructing a Representative Image

Suppose a model classifies images into:

$$\{\text{Class A, Class B, Class C}\}.$$

The attacker starts with a random image x_0 and repeatedly modifies it to maximize the probability of Class A:

$$x_{t+1} = x_t + \eta \nabla_x P_\theta(A \mid x_t).$$

After several iterations:

$$x_0 \rightarrow x_1 \rightarrow \cdots \rightarrow \hat{x}.$$

where

$$\hat{x} = \arg \max_x P_\theta(A \mid x).$$

The resulting image may reveal features that the model associates with Class A.

Interpretation

The attacker is not directly accessing the training images. The attacker is exploiting information encoded in the trained model.

Practical Example: Facial Recognition

Consider a facial recognition system trained using images of individuals.

Face image $\longrightarrow$ identity prediction.

An attacker has access to the prediction system but not the original training images.

The attacker attempts to construct an image $\hat{x}$ that maximizes the probability of a particular identity:

$$\hat{x} = \arg \max_x P_\theta(y^* | x).$$

The reconstructed image may reveal features such as:

- facial structure;
- approximate appearance;
- characteristics strongly associated with the target class.

Privacy implication

A trained model can potentially encode information about sensitive training data.



Model Inversion as an Optimization Problem

Let the model output a probability distribution:

$$f_{\theta}(x) = (P(y_1|x), \dots, P(y_k|x)).$$

For a target class y^* , the attacker can solve:

$$\hat{x} = \arg \max_x P_{\theta}(y^*|x).$$

Equivalently:

$$\hat{x} = \arg \min_x L(\theta, x, y^*).$$

The attacker iteratively updates:

$$x_{t+1} = x_t - \eta \nabla_x L(\theta, x_t, y^*).$$

Security interpretation

The attacker treats the trained model as an oracle and searches for inputs that reveal what the model has learned.

What Information Can Be Inferred?

Model inversion can potentially reveal:

- representative features of a target class;
- sensitive attributes;
- correlations present in training data;
- information about particular individuals in some settings;
- characteristics encoded in model representations.

The severity depends on:

- the type of model;
- the training data;
- how much information the model exposes;
- whether the model overfits;
- the attacker's auxiliary information.

Learned parameters can contain information about training data.

Model Inversion vs. Membership Inference

These two attacks are closely related but ask different questions.

Attack	Question	Information obtained
Membership inference	"Was x in D ?"	Training membership
Model inversion	"What information can I infer?"	Data characteristics

Membership: determine presence

Inversion: infer information

Both can exploit information exposed through model outputs.

Model Inversion vs. Model Extraction

Attack	Attacker wants to learn	Main target
Model extraction	How the model behaves	Model
Membership inference	Whether $x \in D$	Training membership
Model inversion	Information encoded by the model	Training data

Extraction: “What is the model?”
Membership: “Was this record used?”
Inversion: “What can the model reveal?”

Preventing Model Inversion

Common defenses include:

1 Limit model outputs

- Return only necessary information.
- Avoid exposing detailed confidence scores when unnecessary.

2 Reduce overfitting

- Regularization;
- appropriate model complexity;
- careful training.

3 Differential Privacy

- Limit the influence of individual training records.
- Provide a formal privacy guarantee.

4 Access control

- Restrict access to sensitive models.
- Monitor repeated or suspicious queries.

5 Privacy-preserving training

- Use techniques such as secure computation where appropriate.

Cryptographic Perspective

Cryptography can protect training data while it is stored, communicated, or processed.
For example, conceptually:

$$Enc(x) \longrightarrow \boxed{\text{Private computation}} \longrightarrow Enc(f_{\theta}(x)).$$

This can reduce exposure of sensitive inputs during computation.
However:

Encrypted computation $\neq$ automatic protection against inference attacks

If the final model output reveals sensitive information, an attacker may still exploit the output.

Key lesson

Protecting the **data during computation** and limiting what the **model reveals** are different security problems.

Differential Privacy: A Natural Defense

Let D and D' differ in one individual's record:

$$D \simeq D'.$$

A mechanism M is (ϵ, δ) -differentially private if:

$$\Pr[M(D) \in S] \leq e^\epsilon \Pr[M(D') \in S] + \delta.$$

The intuition is:

The presence or absence of one person's data should have limited effect on the output.

This limits the amount of information that an attacker can infer about an individual from the released result.

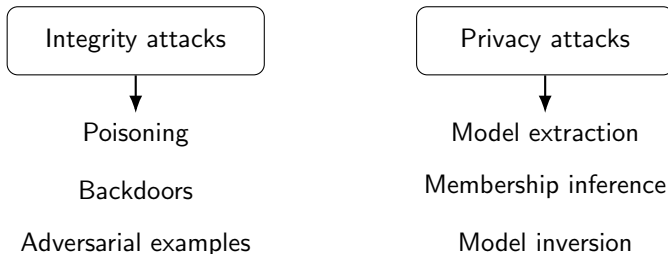
Six Attacks So Far

Attack	Attacked object	Phase	Main goal
Poisoning	Training data	Training	Change model
Backdoor	Training data	Training	Hidden behavior
Adversarial example	Input	Inference	Wrong prediction
Model extraction	Model/API	Inference	Steal model
Membership inference	Training data	Inference	Infer membership
Model inversion	Training information	Inference	Infer information

The ML model itself can become a source of information leakage.

From ML Attacks to Privacy-Preserving ML

We have now seen two broad classes of threats:



Next question

How can we design ML systems so that sensitive data remains private even when the system is being trained or queried?

Privacy-Preserving Machine Learning

Attack 7: Training Data Reconstruction

Attack 7: Training Data Reconstruction

Training-data reconstruction attempts to recover information that is close to the **actual training records** from a trained model or its outputs.

Suppose:

$$D = \{x_1, x_2, \dots, x_n\}$$

is a private training dataset and

$$\theta \leftarrow \text{Train}(D).$$

The attacker observes the trained model f_θ and attempts to construct:

$$\hat{x}_i \approx x_i.$$

Conceptually:

$$D \longrightarrow \theta \longrightarrow f_\theta \longrightarrow \hat{D}$$

Privacy threat

Information from the private training dataset may be recovered from the trained model.

Model Inversion vs. Data Reconstruction

These two attacks are closely related, but the goal is different.

Attack	Attacker tries to obtain	Example
Membership inference	Whether $x \in D$	"Was Alice's record used?"
Model inversion	Characteristics / attributes	"What does a typical member look like?"
Reconstruction	Information close to actual records	"Can I recover training information?"

Reconstruction is a stronger form of privacy leakage.

Caution

Exact recovery of a training record is not always possible. The success of reconstruction depends strongly on the model, training process, and information available to the attacker.

Why Can Training Data Be Recovered?

An ML model is obtained from the training data:

$$\theta = \text{Train}(D).$$

Therefore, the parameters θ may contain information about D .

In particular, reconstruction can become easier when:

- the model memorizes unusual or rare examples;
- the model is over-parameterized;
- the training data is small;
- the model is trained for many iterations;
- the model exposes rich outputs;
- the attacker has auxiliary information.

Important

Generalization and memorization are not identical. A model may generalize well while still memorizing some sensitive information.

Toy Example: Memorized Text

Suppose a language model is trained on private documents.

One training record contains:

“Patient ID: 58321, Diagnosis: ...”

The model is later queried with a suitable prompt.

If the model has memorized the sequence, it may generate information that closely resembles the training text.

Conceptually:

Private document $\longrightarrow$ Training $\longrightarrow$ Model $\longrightarrow$ Generated text

Privacy concern

A model intended to learn general patterns may unintentionally retain and reveal specific information from its training data.

Toy Example: Rare Information

Consider a training dataset containing:

$$D = \{x_1, x_2, \dots, x_{999}, \underbrace{x^*}_{\text{rare record}}\}.$$

The record x^* is highly unusual.

If the model memorizes information specific to x^* , an attacker may exploit model outputs to recover some of that information.

Rare / unique information	→	higher memorization risk
---------------------------	---	--------------------------

This is especially concerning when x^* contains:

- personally identifiable information;
- medical information;
- confidential documents;
- proprietary data.

Practical Example: Sensitive Text Data

Consider a language model trained using:

{emails, reports, messages, documents}.

Some documents contain confidential information.

The attacker interacts with the model through:

Prompt $\longrightarrow$ Language model $\longrightarrow$ Generated response.

The attacker may search for outputs that contain:

- names;
- email addresses;
- phone numbers;
- confidential phrases;
- other memorized information.

Security problem

The model may become an unintended channel through which training data can be disclosed.

Practical Example: Medical Imaging

Consider a model trained on private medical images:

$$D = \{\text{patient medical images}\}.$$

The trained model may encode information about these images.

An attacker attempts to generate or reconstruct an image:

$$\hat{x}$$

that contains information resembling a training image.

$$\text{Private images} \rightarrow \text{Training} \rightarrow \text{Model} \rightarrow \hat{x}$$

Privacy implication

Even when the original images are never directly released, the model can potentially become a source of information about them.

Reconstruction as an Optimization Problem

Let f_θ be the trained model.

The attacker may search for an input that produces a desired model behavior:

$$\hat{x} = \arg \min_x L(f_\theta(x), y^*).$$

More generally, the attacker may optimize:

$$\hat{x} = \arg \min_x [L(f_\theta(x), y^*) + \lambda R(x)],$$

where:

- L measures how well x matches the desired model behavior;
- $R(x)$ imposes constraints or prior information;
- λ controls the trade-off.

The resulting $\hat{x}$ may contain information learned from the training data.

Exact Reconstruction vs. Approximate Reconstruction

It is important to distinguish two cases.

Exact reconstruction

The attacker recovers an actual training record:

$$\hat{x} = x_i.$$

Approximate reconstruction

The attacker recovers an input containing significant information about a training record:

$$\hat{x} \approx x_i.$$

Exact reconstruction is generally much harder.

Privacy leakage does not require exact record recovery.

Even partial recovery of a sensitive attribute can be harmful.

What Makes Reconstruction Easier?

Reconstruction risk can increase when:

- the model memorizes training examples;
- training data contains rare or unique information;
- the model has high capacity;
- the attacker can make many queries;
- model outputs are highly informative;
- the attacker has auxiliary information;
- the model is released publicly.

Thus:

Model capacity + memorization + information-rich outputs $\rightarrow$ privacy risk

Preventing Training Data Reconstruction

Common approaches include:

1 Differential Privacy

- Limit the influence of any individual training record.
- Provide a formal privacy guarantee.

2 Reduce unnecessary memorization

- Regularization;
- appropriate model complexity;
- careful training.

3 Limit model outputs

- Avoid exposing unnecessary information.
- Restrict detailed outputs where possible.

4 Access control

- Restrict access to sensitive models.
- Monitor suspicious queries.

5 Data minimization

- Do not include unnecessary sensitive information in training data.

Cryptographic Perspective

Cryptographic techniques can reduce exposure of training data during storage, communication, and computation.

For example:

$$Enc(D) \longrightarrow \boxed{\text{Private computation}} \longrightarrow \text{Model}.$$

However, protecting the data during computation does not automatically prevent the trained model from leaking information.

Two different problems

Data confidentiality during computation

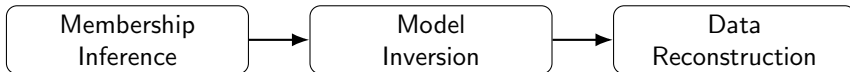
versus

Privacy of information encoded in the trained model.

HE/MPC protect computation; DP limits information leakage.

The Privacy Attack Spectrum

We can now organize the privacy attacks as an increasing level of information disclosure:



Membership: Was x used?

Inversion: What characteristics can be inferred?

Reconstruction: Can training information be recovered?

Common root

The trained model may contain information about its training data.

Seven AI/ML Security Attacks

Attack	Target	Phase	Main goal
Poisoning	Training data	Training	Change model
Backdoor	Training data	Training	Hidden behavior
Adversarial example	Input	Inference	Wrong prediction
Model extraction	Model/API	Inference	Steal model
Membership inference	Training data	Inference	Infer membership
Model inversion	Training information	Inference	Infer attributes
Reconstruction	Training data	Inference	Recover information

AI/ML security = Integrity + Confidentiality + Privacy

Securing AI-ML with Cryptography

Security vs. Privacy in ML

Security asks:

- Can an attacker change the model's behavior?
- Can an attacker make the system unavailable?
- Can an attacker impersonate a trusted participant?

Privacy asks:

- Does the model reveal information about individuals?
- Can training membership be inferred?
- Can sensitive training information be reconstructed?

Important

Security and privacy overlap, but they are not the same problem.

Connecting AI Security to Cryptography

We already know cryptographic tools for protecting:

Data + Communication + Computation

Examples:

Problem	Possible cryptographic approach
Secure communication	Encryption + authentication
Private computation	Homomorphic encryption
Joint computation	Secure MPC
Private statistics / learning	Differential privacy
Distributed private training	Federated learning + secure aggregation
Model/data authenticity	Digital signatures

But cryptography alone does not solve every ML security problem.

Integrity Attacks vs. Privacy Attacks

Integrity Attack

Can the attacker manipulate the ML system?

- Goal: violate correctness or trust
- Changes data, model, or input
- Examples:
 - Data poisoning
 - Backdoor attacks
 - Adversarial examples

Correct system → Manipulated system

Privacy Attack

Can the attacker learn sensitive information?

- Goal: violate confidentiality/privacy
- Extracts information from model/data
- Examples:
 - Membership inference
 - Model inversion
 - Data reconstruction

Private information → Attacker

How Do We Defend Against Them?

Integrity

- Data authentication
- Digital signatures
- Data provenance
- Robust training
- Adversarial training
- Model provenance
- Input validation

Privacy

- Encryption
- Differential Privacy
- Homomorphic Encryption
- Secure MPC
- Secure Aggregation
- Access Control
- Output restriction

Key distinction

Integrity: prevent unauthorized modification
Privacy: prevent unauthorized information leakage

Where Does Cryptography Play a Role?

ML Security Requirement	Cryptographic Tool
Data integrity	Digital signatures, MACs
Data authenticity	Digital signatures, PKI
Data confidentiality	Encryption
Private computation	Homomorphic Encryption
Collaborative private ML	MPC, Secure Aggregation
Model authenticity	Digital signatures, PKI
Secure communication	TLS, authenticated encryption

Cryptography secures the infrastructure around ML

Cryptography Does Not Solve Every ML Attack

Attack	Does Crypto Directly Solve It?
Data poisoning	No, but signatures help authenticate data
Backdoor	No, model provenance helps
Adversarial examples	No, requires robust ML
Model extraction	No, requires API protection
Membership inference	Not directly, DP helps
Model inversion	Not directly, DP/output control helps
Data reconstruction	Not directly, DP helps

Cryptography $\neq$ Complete ML Security

Cryptography protects **data, computation, communication, and trust.**

The Big Picture

$$\text{AI/ML Security} = \text{ML Security} + \text{Cryptographic Security}$$



$$\text{Encryption} + \text{Signatures} + \text{HE} + \text{MPC} + \text{DP}$$

Next: Privacy-Preserving ML $\rightarrow$ Federated Learning $\rightarrow$ Lattice based PQC

Sources:

- 1 Apostol Vassilev, Alina Oprea, Alie Fordyce, Hyrum Anderson, Xander Davies, and Maia Hamin. *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations*. NIST, 2025. NIST AI 100-2e2025. DOI: 10.6028/NIST.AI.100-2e2025.
- 2 Maria Rigaki and Sebastian Garcia. *A Survey of Privacy Attacks in Machine Learning*. ACM Computing Surveys, 56(4):101–134, 2024. DOI: 10.1145/3624010.

Acknowledgment:

- ChatGPT (OpenAI) was used to identify the content for the slides, which was subsequently verified by me.



Dr. Laltu Sardar, Assistant Professor, IISER Thiruvananthapuram
laltu.sardar@iisertvm.ac.in, laltu.sardar.crypto@gmail.com
Course webpage: https://laltu-sardar.github.io/courses/26_crypto.html.